

TOP 50 BACKEND INTERVIEW QUESTIONS

☆ Page 1 - Backend Fundamentals (Q1 - Q10) ☆

1. What is a REST API?

Ans: REST (Representational State Transfer) API is an architectural style that uses HTTP methods to perform CRUD operations on resources using URLs.

CRUD with REST

GET	→ Read
POST	→ Create
PUT	→ Update
DELETE	→ Delete

2. REST vs SOAP?

Ans: REST is lightweight, uses JSON, HTTP, and is easy to use. SOAP is protocol-based, uses XML, has strict standards and supports advanced security.

	REST	SOAP
Protocol	HTTP	HTTP / SMTP
Data Format	JSON	XML
Speed	Fast	Slow
Complexity	Low	High

3. What are HTTP methods?

Ans: GET, POST, PUT, PATCH, DELETE, OPTIONS, HEAD.

4. Common HTTP status codes?

Ans: 2xx (Success) → 200 OK, 201 Created

3xx (Redirection) → 301 Moved, 304 Not Modified

4xx (Client Error) → 400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found

5xx (Server Error) → 500 Internal Server Error, 502 Bad Gateway

5. PUT vs PATCH?

Ans: PUT → Replaces the entire resource.

PATCH → Updates only the provided fields/partial update.



6. What is idempotency?

Ans: An operation is idempotent if multiple identical requests have the same effect as a single request. PUT and DELETE are idempotent.

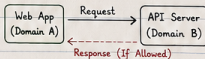
7. Stateless vs Stateful?

Ans: Stateless → Server does not store client state. Each request is independent. (REST APIs)

Stateful → Server stores client state (e.g., Session-based authentication).

8. What is CORS?

Ans: CORS (Cross-Origin Resource Sharing) is a security feature that allows or restricts web pages from making requests to a different domain.



9. What is API versioning?

Ans: API versioning is a way to manage changes in APIs without breaking existing clients. Example: /api/v1/users

Ways to Version:

- URL (/v1/users)
- Header (Accept-Version)
- Query Param (?version=1)

10. What is Content Negotiation?

Ans: It is the process where the server returns a response in the format (JSON, XML, etc.) that the client prefers using headers like Accept: application/json

Most Asked:

- ★ REST vs SOAP
- ★ PUT vs PATCH
- ★ Idempotency



Pro Tip: Always follow REST conventions, use proper status codes and keep your APIs stateless for better scalability.

TOP 50 BACKEND INTERVIEW QUESTIONS

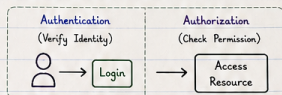


Page 2 - Authentication & API Security (Q11-Q20)



11. Authentication vs Authorization ?

Ans: Authentication verifies who you are.
Authorization determines what you can do.



12. JWT vs Session ?

Ans: JWT is stateless and stored on client (usually in localStorage or cookies).
Session is stateful and stored on server.

Feature	JWT	Session
Storage	Client (Token)	Server (Session Store)
State	Stateless	Stateful
Scalability	High	Low (depends on store)
Performance	Fast	Relatively Slow
Security	Secure (if implemented right)	Secure

13. OAuth vs JWT ?

Ans: OAuth is an authorization framework that allows access to resources without sharing credentials.
JWT is a token format used to securely transmit information between parties.

OAuth (Protocol / Framework)

- Used for delegated authorization
- Example: Google Login, GitHub Login
- Involves Authorization Server

JWT (Token)

- Used to represent claims
- Compact and self-contained
- Used in OAuth, Custom Auth

14. Access Token vs Refresh Token ?

Ans: Access Token is short-lived and used to access resources.
Refresh Token is long-lived and used to get a new Access Token.

Feature	Access Token	Refresh Token
Purpose	Access resources	Get new access token
Lifetime	Short (5-15 min)	Long (7-30 days)
If Stolen	Damage is limited	Can issue new access token
Storage	Memory / LocalStorage	HttpOnly Secure Cookie

15. What is HTTPS ?

Ans: HTTPS is HTTP over TLS/SSL. It encrypts data between client and server to ensure confidentiality, integrity and authenticity.



16. What is CORS ?

Ans: CORS (Cross-Origin Resource Sharing) allows a server to specify which origins (domains) are allowed to access its resources.



17. CSRF vs XSS ?

Ans: CSRF (Cross-Site Request Forgery): Attacker tricks user browser to make unwanted requests.
XSS (Cross-Site Scripting): Attacker injects malicious scripts that run in user's browser.

CSRF	XSS
• Targets authenticated users	• Targets other users
• Uses user's session/cookies	• Injects malicious script
• Example: Transfer money	• Example: Steal Cookies

18. What is SQL Injection ?

Ans: SQL Injection is an attack where malicious SQL queries are inserted via input fields to manipulate the database.

Unsafe Query:
`SELECT * FROM users WHERE name = '' + userInput + ''`
Safe Approach:
 Use Prepared Statements / Parameterized Queries

19. What is Rate Limiting ?

Ans: Rate Limiting restricts the number of requests a client can make within a specific time window to prevent abuse.

Example: 100 requests / 15 minutes / IP

20. API Keys vs OAuth ?

Ans: API Keys are simple tokens used for identifying the client.
OAuth is more secure and used for delegated access with user consent.

Feature	API Key	OAuth
Purpose	Identify Client	Authorize Access
Security	Less Secure	More Secure
User Involvement	No	Yes (Consent)
User Case	Simple Public APIs	User Data / Third-party Access



Pro Tip

Never store tokens in localStorage. Use HttpOnly, Secure Cookies for Refresh Tokens.
Always validate and sanitize all inputs.



Most Asked

- JWT vs Session
- Access vs Refresh Token
- SQL Injection
- CSRF vs XSS

Remember

Security is not a feature,
it's a habit. 😊

TOP 50 BACKEND INTERVIEW QUESTIONS

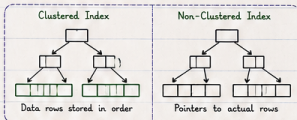


Page 3 - Database & SQL (Q21 - Q30)



21. What is the difference between Clustered and Non-Clustered Index ?

- Ans:**
- Clustered Index → Sorts and stores the actual data rows. Only one per table.
 - Non-Clustered Index → Separate structure with a pointer to actual data rows. Multiple allowed.



22. What is Indexing ?

- Ans:** Indexing is a technique to improve the speed of data retrieval operations on a database table. It creates a data structure that allows fast lookups.

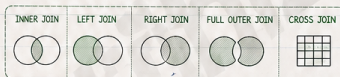
Benefits:

- Faster SELECT queries
- Efficient sorting and filtering
- Improves overall performance



23. What are the types of SQL JOINS ?

- Ans:** INNER JOIN, LEFT JOIN, RIGHT JOIN, FULL OUTER JOIN, CROSS JOIN.



24. What is the difference between INNER JOIN and LEFT JOIN ?

- Ans:** INNER JOIN returns only matching rows from both tables. LEFT JOIN returns all rows from the left table and matching rows from the right table. If no match, NULLs are returned for right table columns.

INNER JOIN Example			LEFT JOIN Example		
A.id	A.name	B.name	A.id	A.name	B.name
1	John	Sales	1	John	Sales
2	Alice	HR	2	Alice	HR
			3	Bob	NULL

25. What are the ACID properties ?

- Ans:** Atomicity, Consistency, Isolation, Durability. They ensure reliable processing of database transactions.

Atomicity	Consistency	Isolation	Durability
All or nothing execution	Database remains in valid state	Transactions do not interfere with each other	Committed data is permanent

26. What is a Transaction ?

- Ans:** A transaction is a sequence of operations performed as a single logical unit of work. It follows ACID properties.

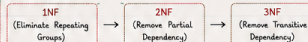
Example: Money Transfer

Debit A/c → Credit B/a → Commit

27. What is Normalization ?

- Ans:** Normalization is the process of organizing data in a database to reduce redundancy and improve data integrity.

Normal Forms:



28. What is a Stored Procedure ?

- Ans:** A stored procedure is a precompiled collection of SQL statements stored in the database. It can accept parameters.

```
CREATE PROCEDURE GetUsersByCity(IN cityName VARCHAR(50))
BEGIN
  SELECT * FROM Users WHERE city = cityName;
END;
```

29. What is the difference between View and Table ?

- Ans:** View → Virtual table based on query result. No data stored. Table → Actual data is stored physically.

Feature	View	Table
Storage	No (Virtual)	Yes (Physical)
Data	From other tables	Stores actual data
Performance	Slower (depends)	Faster
Modifiable	Limited	Yes

30. How do you optimize an SQL query ?

- Ans:** Use Indexing, Avoid SELECT *, Use proper WHERE clause, Optimize JOINS, Use LIMIT, Avoid functions on indexed columns, Use EXPLAIN to analyze.

Quick Tips

- ✓ Create indexes on frequently searched columns.
- ✓ Use specific columns instead of SELECT *
- ✓ Avoid unnecessary subqueries.
- ✓ Analyze query using EXPLAIN / ANALYZE.



Pro Tip

Always analyze slow queries and index only what is necessary. Over-indexing can also degrade performance.



Most Asked

- Indexing
- JOINS
- Normalization
- Transactions (ACID)

Remember

Good database design is the foundation of a scalable application.



TOP 50 BACKEND INTERVIEW QUESTIONS

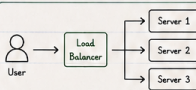


Page 4 - System Design & Scalability (Q31-Q40)



31. What is a Load Balancer?

Ans: Load Balancer distributes incoming traffic across multiple servers to ensure no single server is overloaded and improve availability and reliability.



Benefits:

- High Availability
- Better Performance
- Fault Tolerance
- Scalability

32. What is CDN?

Ans: CDN (Content Delivery Network) is a network of servers distributed globally that deliver content to users from the nearest location.

How CDN works?

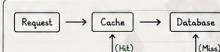


Benefits:

- Faster content delivery
- Reduced latency
- Reduced load on origin
- Better user experience

33. What is Caching?

Ans: Caching stores frequently accessed data in memory so that future requests are served faster.



Benefits:

- Faster Response
- Reduced DB Load
- Better Scalability

34. What is Redis used for?

Ans: Redis is an in-memory data structure store used for caching, session storage, rate limiting, leaderboards, queues and more.

Common Use Cases:



35. What is a Message Queue?

Ans: Message Queue is a way for applications to communicate asynchronously by sending messages through a queue.



Benefits:

- Decouples services
- Handles spikes
- Improves reliability

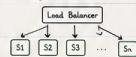
36. Microservices vs Monolith?

	Monolith	Microservices
Architecture	Single deployable unit	Small independent services
Deployment	Deploy all together	Deploy independent services
Scaling	Scale entire application	Scale individual services
Fault Isolation	Low	High
Technology	Same for entire app	Different per service
Complexity	Simple	Complex

37. Horizontal vs Vertical Scaling?

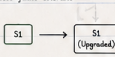
Horizontal Scaling (Scale Out)

- Add more machines/instances
- Distribute load
- Cost-effective
- More fault tolerant



Vertical Scaling (Scale Up)

- Increase power of existing machine
- More CPU / RAM
- Limited by hardware
- Less fault tolerant



38. What is Database Sharding?

Ans: Sharding is a technique to split large database into smaller parts called shards and distribute across multiple servers.

Sharding Types:

1. Horizontal Sharding → Split rows across servers
2. Vertical Sharding → Split columns across servers



39. What is CAP Theorem?

Ans: In a distributed system, you can have at most following three guarantees:

Consistency (C)

All nodes see the same data at the same time.

Availability (A)

System remains available for every request.

Partition Tolerance (P)

System continues to work despite network partitions or failures.

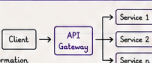


40. What is an API Gateway?

Ans: API Gateway is a single entry point for all client requests. It routes requests to appropriate services and handles cross-cutting concerns.

Responsibilities:

- Routing
- Authentication & Authorization
- Rate Limiting
- Logging & Monitoring
- Caching
- Request/Response Transformation



Pro Tip

Always design for failure.
Scalability + Monitoring +
Automation = Reliable Systems.



Most Asked

- Caching
- Load Balancer
- Database Sharding
- Microservices vs Monolith

Remember

Good system design is not about the tools, it's about trade-offs and priorities.



TOP 50 BACKEND INTERVIEW QUESTIONS

☆ Page 5 - Backend Architecture & Best Practices (Q41-Q50) ☆

41. What are SOLID Principles?

Ans: SOLID is a set of five design principles that make code more maintainable, scalable and easy to extend.

SOLID Principles

- S** Single Responsibility Principle
- O** Open/Closed Principle
- L** Liskov Substitution Principle
- I** Interface Segregation Principle
- D** Dependency Inversion Principle

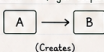
💡 Why Important?

- Easy to maintain
- Easy to test
- Scalable & flexible
- Fewer bugs

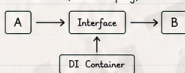
42. What is Dependency Injection?

Ans: Dependency Injection is a design pattern where the dependencies of a class are provided by an external source rather than the class creating them itself.

Without DI (Tight Coupling)



With DI (Loose Coupling)



43. What are Design Patterns?

Ans: Design Patterns are reusable solutions to commonly occurring problems in software design. They help write clean, maintainable and scalable code.

Commonly Used Patterns

- **Singleton** : Ensure a class has only one instance.
- **Factory** : Create objects without exposing instantiation logic.
- **Repository** : Abstraction layer over data access logic.
- **Builder** : Step-by-step construction of complex objects.
- **Observer** : One-to-many dependency between objects.

44. What is Repository Pattern?

Ans: Repository Pattern acts as an abstraction layer between the business logic and data access layer. It helps in cleaner code and easier testing.



45. What is Middleware?

Ans: Middleware is a function that has access to the request and response object in the pipeline. It can execute code, make changes, and the request-response cycle or call the next middleware.



46. What is Logging and why is it important?

Ans: Logging is the process of recording events that occur in an application. It helps in debugging, monitoring and auditing.

Log Levels

- **DEBUG** : Detailed information
- **INFO** : General operational messages
- **WARN** : Something unexpected but not serious
- **ERROR** : Something failed
- **FATAL** : Critical errors causing shutdown



47. What is Exception Handling?

Ans: Exception Handling is a way to handle runtime errors gracefully without crashing the application.

Try-Catch Example (Most Languages)

```

try {
  // risky code
} catch (Exception e) {
  // handle exception
} finally {
  // cleanup code
}
  
```

Best Practices

- Don't swallow exceptions
- Log the exception
- Return meaningful error messages
- Handle specific exceptions



48. What is Docker?

Ans: Docker is a containerization platform that packages an application and its dependencies into a container so it can run consistently anywhere.



49. What is CI/CD Pipeline?

Ans: CI/CD stands for Continuous Integration and Continuous Deployment. It automates building, testing and deploying applications.



50. Explain a project architecture you built.

Ans: Explain the high level architecture, tech stack, modules, database design, APIs, integrations, challenges faced and how you solved them.

How to Answer? (Structure)

1. Problem Statement
2. Solution / Architecture Overview
3. Tech Stack & Why

4. Key Modules
5. Database Design
6. Challenges & Solutions
7. Results / Impact



Pro Tip

Give real examples from your project while answering. It makes your answer strong and memorable.



Most Asked

- System Design Concepts
- Database & SQL
- Caching & Scalability
- Security Topics

Remember

It's not about memorizing answers, it's about understanding concepts and applying them.

